ReactJS

Unit 5 — Routing
Navigation · Routes · Link · useParams

In This Unit

1. What is routing?
2. React Router setup
3. Routes and Links
4. NavLink for menus
5. Route parameters
6. Nested routes

Continuation…

7 useEffect Hook intro

8 Dependency arrays

9 Cleanup functions

10 API calls with useEffect

ROUTING BASICS

Navigation in SPAs

What is Routing?

- Routing = navigating between different pages/views without reloading
- React is a Single Page Application (SPA)
- React Router is the standard library for routing

Install React Router

```
npm install react-router-dom
```

Why Not Use <a> Tags?

- Using <a href="/about"> causes a full page reload
- This re-downloads all CSS and JavaScript
- State is lost, everything re-initializes

Don't do this

```
<a href="/about">About</a>  {/* full reload
*/}
```

Use Link instead

```
<Link to="/about">About</Link>
```

React Router Setup

```
import { BrowserRouter, Routes, Route,
         Link } from 'react-router-dom';

function App() {
  return (
    <BrowserRouter>
      <Routes>
        <Route path="/" element={<Home />} />
        <Route path="/about" element={<About />} />
      </Routes>
    </BrowserRouter>
  );
}
```

Three Key Components

BrowserRouter

Wraps the entire app

Routes

Container for all routes

Route

Maps path to component

NAVIGATION

Link · NavLink

Link Component

```
import { Link } from 'react-router-dom';

<Link to="/">Home</Link>
<Link to="/about">About</Link>
<Link to="/contact">Contact</Link>
```

- Use to prop instead of href
- No page reload
- Smooth navigation in SPA

Navigation Bar with Link

```
function App() {
  return (
    <BrowserRouter>
      <div>
        <Link to="/">Home</Link> |
        <Link to="/about">About</Link> |
        <Link to="/contact">Contact</Link>
      </div>
      <Routes>
        <Route path="/" element={<Home />} />
        <Route path="/about" element={<About />} />
        <Route path="/contact" element={<Contact />} />
      </Routes>
    </BrowserRouter>
  );
}
```

NavLink for Styling

NavLink

```
import { NavLink } from 'react-router-dom';

<NavLink to="/about">About</NavLink>
```

- NavLink = Link with styling
- Adds active class to current page
- Better for navigation menus

Styling Active Links

```
<NavLink to="/about"
  className={({isActive}) =>
    isActive ? 'active' : ''
  }
>
  About
</NavLink>

/* CSS */
.active { color: red; }
```

DYNAMIC ROUTES

Route parameters · useParams

Route Parameters

- Use :paramName in the path
- Example: /student/:id
- Allows dynamic pages based on data

```
<Route path="/student/:id"
      element={<Student />} />
```

Reading Route Parameters

```
import { useParams } from 'react-router-dom';

function Student() {
  const { id } = useParams();

  return (
    <div>
      <h1>Student ID: {id}</h1>
    </div>
  );
}
```

- Use useParams() Hook
- Extract parameters from the URL

Linking to Dynamic Routes

Passing params in links

```
<Link to="/student/1">Ravi</Link>
<Link to="/student/2">Kiran</Link>
<Link to="/student/3">Anil</Link>
```

Read in component

```
// In Student component
const { id } = useParams();
console.log(id);  // 1, 2, or 3
```

The id is extracted from the URL and can be used to fetch data.

Example: Student List and Detail

```
const students = [
  {id: 1, name: 'Ravi'},
  {id: 2, name: 'Kiran'},
  {id: 3, name: 'Anil'}
];

{students.map(s => (
  <Link key={s.id} to={`/student/${s.id}`}>
    {s.name}
  </Link>
))}
```

NESTED ROUTES

Outlet · Layout routes

Nested Routes: Layout Pattern

- A parent route can have child routes
- Useful for layout: sidebar + content
- Child content renders in an Outlet

```
<Route path="/dashboard" element={<Dashboard />}>
  <Route path="profile" element={<Profile />} />
  <Route path="settings" element={<Settings />} />
</Route>
```

Using Outlet

```
import { Outlet } from 'react-router-dom';

function Dashboard() {
  return (
    <div>
      <h1>Dashboard</h1>
      <nav>
        <Link to="profile">Profile</Link>
        <Link to="settings">Settings</Link>
      </nav>
      <Outlet />  {/* child route renders here */}
    </div>
  );
}
```

Complete Nested Routing Example

```
<BrowserRouter>
  <Routes>
    <Route path="/" element={<App />}>
      <Route path="dashboard" element={<Dashboard />}>
        <Route path="profile" element={<Profile />} />
        <Route path="settings" element={<Settings />} />
      </Route>
      <Route path="home" element={<Home />} />
    </Route>
  </Routes>
</BrowserRouter>
```

Practice

- Create routes for Home, About, Contact pages
- Add Link components for navigation
- Create a /student/:id route
- Use useParams() to read the student ID
- Build a student list with links to detail pages

Summary

- Routing enables navigation without page reloads in SPAs
- Wrap app in BrowserRouter
- Define routes with Routes and Route
- Use Link for navigation, NavLink for menus
- Use useParams() to read :parameters from URLs
- Use Outlet for nested routes

USEEFFECT

Side effects after render

What are Side Effects?

- A side effect = something that happens after the component renders
- Examples: fetch data from an API, update document title, subscribe to events

useEffect

Hook that runs side effects after render

When it runs

After React updates the screen

useEffect Syntax

- useEffect accepts two arguments:
- 1. A function with your side effect code
- 2. A dependency array (controls when it runs)

```
import { useEffect } from 'react';

useEffect(() => {
  // Your side effect code here
}, []);
```

Runs After Every Render

No dependency array

```
useEffect(() => {
  console.log('After render');
});
```

- Runs after every render
- Useful for watching state changes

Runs Once: Empty Dependency Array

Empty []

```
useEffect(() => {
  console.log('Once, on mount');
}, []);
```

- Runs only once, when component mounts
- Use for loading data or initializing

Runs When Dependency Changes

With dependency

```
useEffect(() => {
  console.log('count changed');
}, [count]);
```

- Runs when count changes
- Also runs on first render

Dependency Array Comparison

Form	When it runs
useEffect(fn)	After every render
useEffect(fn, [])	Once, on mount
useEffect(fn, [x, y])	On mount, and when x or y changes

CLEANUP & API

Cleanup functions · Fetching data

Cleanup Function

```
useEffect(() => {
  const id = setInterval(() => {
    console.log('tick');
  }, 1000);

  return () => clearInterval(id);
}, []);
```

- return a function from useEffect to clean up
- Runs when component unmounts
- Stops timers, listeners, subscriptions

Fetching Data with useEffect

```
useEffect(() => {
  fetch('/api/students')
    .then(r => r.json())
    .then(data => setStudents(data))
    .catch(err => console.error(err));
}, []);
```

- Use useEffect to fetch API data
- Empty [] = fetch once on mount
- Store result in state with setState

Loading State Pattern

```
const [loading, setLoading] = useState(true);

useEffect(() => {
  fetch('/api/students')
    .then(r => r.json())
    .then(data => {
      setStudents(data);
      setLoading(false);
    });
}, []);
```

- Track loading state
- Show 'Loading...' while fetching
- Show data when done

Practice

- Render a list of items with map()
- Add keys to the list
- Fetch data from an API with useEffect
- Show loading state while fetching
- Display the data when done

Summary

- map() transforms arrays into lists of elements
- Keys help React track items and optimize re-renders
- useEffect runs side effects after the component renders
- Dependency array controls when useEffect runs
- Return a cleanup function to stop timers and listeners